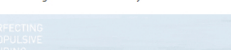


Falcon 9 first stage will land successfully

PERFECTING PROPULSIVE LANDING

A photograph showing a Falcon 9 first stage landing on a grassy field. The stage is positioned vertically, and a large, billowing plume of orange smoke and fire is visible at its base, indicating the landing burn. The background shows a green field and a blue sky.

Flight No.	Date and time (UTC)	Version, Booster ^[4]	Launch site	Payload ^[4]	Payload mass	Orbit	Customer	Launch outcome	Booster landing
78	7 January 2020, 02:19:21 ^[490]	F9 B5 △ F1049.4	CCAFS, SLC-40	Starlink 2 v1.0 (60 satellites)	15,600 kg (34,400 lb) ^[5]	LEO	SpaceX	Success	Success (dome ship)
	Third large batch and second operational flight of Starlink constellation. One of the 60 satellites included a test coating to make the satellite less reflective, and thus less likely to interfere with ground-based astronomical observations. ^[490]								
	19 January 2020, 15:30 ^[491]	F9 B5 △ F1046.4	KSC, LC-39A	Crew Dragon in-flight abort test ^[492] (Dragon C205.1)	12,050 kg (26,570 lb)	Sub-orbital ^[496]	NASA (CTS) ^[497]	Success	No attempt
79	An atmospheric test of the Dragon 2 abort system after Max Q. The capsule fired the SuperDraco engines, reached an apogee of 40 km (25 mi), deployed parachutes after reentry, and splashed down in the ocean 31 km (19 mi) downrange from the launch site. The test was previously slated to be accomplished with the Crew Dragon Demo-1 capsule ^[498] but that test article exploded during a ground test of SuperDraco engines on 20 April 2019. ^[1416] The abort test used the capsule originally intended for the first crew flight. ^[499] As expected, the booster was destroyed by aerodynamic forces after the capsule abort. ^[500] First flight of a Falcon 9 with only one functional stage – the second stage had a mass simulator in place of its engine.								
80	29 January 2020, 14:02 ^[501]	F9 B5 △ F1029.3	CCAFS, SLC-40	Starlink 3 v1.0 (60 satellites)	15,600 kg (34,400 lb) ^[5]	LEO	SpaceX	Success	Success (dome ship)
	Third operational and fourth large batch of Starlink satellites, deployed in a circular 290 km (180 mi) orbit. One of the failing halves was caught, while the other was fished out of the ocean. ^[502]								
	17 February 2020, 15:05 ^[503]	F9 B5 △ F1056.4	CCAFS, SLC-40	Starlink 4 v1.0 (60 satellites)	15,600 kg (34,400 lb) ^[5]	LEO	SpaceX	Success	Failure (dome ship)
81	Fourth operational and fifth large batch of Starlink satellites. Used a new flight profile which deployed into a 212 km × 386 km (132 mi × 240 mi) elliptical orbit instead of launching into a circular orbit and firing the second stage engine twice. The first stage booster failed to land on the drone ship ^[504] due to incorrect wind data. ^[505] This was the first time a flight proven booster failed to land.								
	7 March 2020, 04:50 ^[506]	F9 B5 △ F1059.2	CCAFS, SLC-40	SpaceX CRS-20 (Dragon C112.3 △)	1,977 kg (4,359 lb) ^[507]	LEO (ISS)	NASA (CRS)	Success	Success (ground pad)
82	Last launch of phase 1 of the CRS contract. Carries Bartolomeo, an ESA platform for hosting external payloads onto ISS. ^[508] Originally scheduled to launch on 2 March 2020, the launch date was pushed back due to a second stage engine failure. SpaceX decided to swap out the second stage instead of replacing the faulty part. ^[509] It was SpaceX's 50th successful landing of a first stage booster, the third flight of the Dragon C112 and the last launch of the cargo Dragon spacecraft.								
	18 March 2020, 12:16 ^[510]	F9 B5 △ F1048.5	KSC, LC-39A	Starlink 5 v1.0 (60 satellites)	15,600 kg (34,400 lb) ^[5]	LEO	SpaceX	Success	Failure (dome ship)
83	Fifth operational launch of Starlink satellites. It was the first time a first stage booster flew for a fifth time and the second time the fairings were reused (Starlink flight in May 2019). ^[511] Towards the end of the first stage burn, the booster suffered premature shut down of an engine, the first of a Merlin 1D variant and first since the CRS-1 mission in October 2012. However, the payload still reached the targeted orbit. ^[512] This was the second Starlink launch booster landing failure in a row, later revealed to be caused by residual cleaning fluid trapped inside a sensor. ^[513]								
84	22 April 2020, 09:35 ^[514]	F9 B5 △ F1051.4	KSC, LC-39A	Starlink 6 v1.0 (60 satellites)	15,600 kg (34,400 lb) ^[5]	LEO	SpaceX	Success	Success (dome ship)

Objectives

Web scrap Falcon 9 launch records with BeautifulSoup :

- Extract a Falcon 9 launch records HTML table from Wikipedia
- Parse the table and convert it into a Pandas data frame

First let's import required packages for this lab

```
import sys

import requests
from bs4 import BeautifulSoup
import re
import unicodedata
import pandas as pd

and we will provide some helper functions for you to process web scraped HTML table

def date_time(table_cells):
    """
    This function returns the data and time from the HTML table cell
    Input: the element of a table data cell extracts extra row
    """
    return [data_time.strip() for data_time in list(table_cells.strings)][0:2]

def booster_version(table_cells):
    """
    This function returns the booster version from the HTML table cell
    Input: the element of a table data cell extracts extra row
    """
    out = ''
    for i in range(1, len(table_cells.strings)):
        out += '%s' % (booster_version_in_string % (table_cells.strings[i]))
    return out

def landing_status(table_cells):
    """
    This function returns the landing status from the HTML table cell
    Input: the element of a table data cell extracts extra row
    """
    out = ''
    for i in range(1, len(table_cells.strings)):
        out += '%s' % (landing_status_in_string % (table_cells.strings[i]))
    return out

def get_mass(table_cells):
    mass = unicodedata.normalize('NFKD', table_cells.text).strip()
    if mass:
        mass.find("kg")
        new_mass = mass[0:mass.find("kg")-2]
    else:
        new_mass = 0
    return new_mass

def extract_column_from_header(row):
    """
    This function returns the landing status from the HTML table cell
    Input: the element of a table data cell extracts extra row
    """
    if (row.br):
        row.br.extract()
    if row.a:
        row.a.extract()
    if row.sup:
        row.sup.extract()
    row.sup.extract()
```

To keep the lab tasks consistent, you will be asked to scrape the data from a snapshot of the [List of Falcon 9 and Falcon Heavy launches](#). Wikipage updated on 9th June 2021

```
static_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922"
```

Next, request the HTML page from the above URL and get a `response` object

TASK 1: Request the Falcon9 Launch Wiki page from its URL

First, let's perform an HTTP GET method to request the Falcon9 Launch HTML page, as an HTTP response.

```
[ ]: # Use requests.get() method with the provided static_url
# assign the response to a object
```

Create a BeautifulSoup object from the HTML response

```
[ ]: # Use BeautifulSoup() to create a BeautifulSoup object from a response text content
```

Print the page title to verify if the BeautifulSoup object was created properly

```
[ ]: # Use soup.title attribute
```

TASK 2: Extract all column/variable names from the HTML table header

Next, we want to collect all relevant column names from the HTML table header

Let's try to find all tables on the wiki page first. If you need to refresh your memory about BeautifulSoup, please check the external reference link towards the end of this lab

```
[ ]: # Use the find_all function in the BeautifulSoup object, with element type 'table'.
# Assign the result to a list called 'html_tables'
```

Starting from the third table is our target table contains the actual launch records.

```
[ ]: # Let's print the third table and check its content
first_launch_table = html_tables[2]
print(first_launch_table)
```

You should be able to see the columns names embedded in the table header elements <th> as follows:

```
<tr>
<th scope="col">Flight No.
</th>
<th scope="col">Date and<br/>Time (<a href="/wiki/Coordinated Universal Time" title="Coordinated Universal Time">UTC</a>)
</th>
<th scope="col"><a href="/wiki/List_of_Falcon_9_first-stage_boosters" title="List of Falcon 9 first-stage boosters">Version,<br/>Booster</a> <sup class="reference" id="cite_ref-booster-11-0"><a href="#cite_note-booster-11">[b]</a>
</th>
</tr>
# Assign the result to a list called 'html_tables'
```

Would you like to receive official Jupyter news?
Please read the privacy policy.
[Open privacy policy](#) Yes No

Starting from the third table is our target table contains the actual launch records.

```
[ ]: # Let's print the third table and check its content
first_launch_table = html_tables[2]
print(first_launch_table)
```

You should be able to see the columns names embedded in the table header elements <th> as follows:

```
<tr>
<th scope="col">Flight No.
</th>
<th scope="col">Date and<br/>Time (<a href="/wiki/Coordinated Universal Time" title="Coordinated Universal Time">UTC</a>)
</th>
<th scope="col"><a href="/wiki/List_of_Falcon_9_first-stage_boosters" title="List of Falcon 9 first-stage boosters">Version,<br/>Booster</a> <sup class="reference" id="cite_ref-booster-11-0"><a href="#cite_note-booster-11">[b]</a></sup>
</th>
<th scope="col">Launch site
</th>
<th scope="col">Payload<sup class="reference" id="cite_ref-Dragon-12-0"><a href="#cite_note-Dragon-12">[c]</a></sup>
</th>
<th scope="col">Payload mass
</th>
<th scope="col">Orbit
</th>
<th scope="col">Customer
</th>
<th scope="col">Launch<br/>Outcome
</th>
<th scope="col"><a href="/wiki/Falcon_9_first-stage_landing_tests" title="Falcon 9 first-stage landing tests">Booster<br/>Landing</a>
</th>
</tr>
```

Next, we just need to iterate through the <th> elements and apply the provided 'extract_column_from_header()' to extract column name one by one

```
[ ]: column_names = []
# Apply find_all() function with 'th' element on first_launch_table
# Iterate each th element and apply the provided extract_column_from_header() to get a column name
# Append the non-empty column name ('if name is not None and len(name) > 0') into a list called column_names
```

Check the extracted column names

```
[ ]: print(column_names)
```

TASK 3: Create a data frame by parsing the launch HTML tables

We will create an empty dictionary with keys from the extracted column names in the previous task. Later, this dictionary will be converted into a Pandas dataframe

usually, HTML tables in wiki pages are likely to contain unexpected annotations and other types or noises, such as reference links, missing values, N/A (e.g., inconsistent formatting, etc.

To simplify the parsing process, we have provided an incomplete code snippet below to help you to fill up the 'launch_dict'. Please complete the following code snippet with TODOs or you can choose to write your own logic to parse all launch tables:

```
[ ]: extracted_row = 0
# Extract each table
for table_number, table in enumerate(soup.find_all('table', "wiki-table plainrowheaders collapsible")):
    # get Table row
    for rows in table.find_all('tr'):
        # check to see if first table heading is as number corresponding to launch number
        if rows.th:
            if rows.th.string:
                flight_number_rows.th.string.strip()
                flag=flight_number.isdigit()
            else:
                flag=False
            # get table element
            row=soup.find_all('td')
            # if it is a number, add it to a dictionary
            if flag:
                extracted_row += 1
                # Flight Number value
                # TODO: Append the flight_number into launch_dict with key 'Flight No.'
                # print(flight_number)
                datatimelist[date_time(row[0])]
            # Date value
            # TODO: Append the date into launch_dict with key 'Date'
            date = datatimelist[0].strip(',')
            # print(date)
            # Time value
            # TODO: Append the time into launch_dict with key 'Time'
            time = datatimelist[1]
            # print(time)
            # Booster version
            # TODO: Append the bv into launch_dict with key 'Version Booster'
            bv=booster_version(row[1])
            if not(bv):
                bv=row[1].a.string
                print(bv)
            # Launch Site
            # TODO: Append the bv into launch_dict with key 'Launch Site'
            launch_site = row[2].a.string
            # print(launch_site)
            # Payload
            # TODO: Append the payload into launch_dict with key 'Payload'
            payload = row[3].a.string
            # print(payload)
            # Payload Mass
            # TODO: Append the payload_mass into launch_dict with key 'Payload Mass'
            payload_mass = get_mass(row[4])
            # print(payload_mass)
            flag=flight_number.isdigit()
            else:
                flag=False
            # get table element
            row=soup.find_all('td')
            # if it is a number, add it to a dictionary
            if flag:
                extracted_row += 1
                # Flight Number value
                # TODO: Append the flight_number into launch_dict with key 'Flight No.'
                # print(flight_number)
                datatimelist[date_time(row[0])]
            # Date value
            # TODO: Append the date into launch_dict with key 'Date'
            date = datatimelist[0].strip(',')
            # print(date)
            # Time value
            # TODO: Append the time into launch_dict with key 'Time'
            time = datatimelist[1]
            # print(time)
            # Booster version
            # TODO: Append the bv into launch_dict with key 'Version Booster'
            bv=booster_version(row[1])
            if not(bv):
                bv=row[1].a.string
                print(bv)
            # Launch Site
            # TODO: Append the bv into launch_dict with key 'Launch Site'
            launch_site = row[2].a.string
            # print(launch_site)
```

Would you like to receive official Jupyter news?
Please read the privacy policy.
[Open privacy policy](#) Yes No

Would you like to receive official Jupyter news?
Please read the privacy policy.
[Open privacy policy](#) Yes No

```
-----
# Payload
# TODO: Append the payload into launch_dict with key 'Payload'
payload = row[3].a.string
#print(payload)

# Payload Mass
# TODO: Append the payload_mass into launch_dict with key 'Payload mass'.
payload_mass = get_mass(row[4])
#print(payload_mass)

# Orbit
# TODO: Append the orbit into launch_dict with key 'Orbit'
orbit = row[5].a.string
#print(orbit)

# Customer
# TODO: Append the customer into launch_dict with key 'Customer'
customer = row[6].a.string
#print(customer)

# Launch outcome
# TODO: Append the launch_outcome into launch_dict with key 'Launch outcome'.
launch_outcome = list(row[7].strings)[0]
#print(launch_outcome)
# TODO: Append the launch_outcome into launch_dict with key 'Launch outcome'.
launch_outcome = list(row[7].strings)[0]
#print(launch_outcome)

# Booster Landing
# TODO: Append the launch_outcome into launch_dict with key 'Booster Landing'.
booster_landing = landing_status(row[8])
#print(booster_landing)
-----
```

After you have fill in the parsed launch record values into `launch_dict`, you can create a dataframe from it.

```
[ ]: df=pd.DataFrame(L_key:pd.Series(value) for key, value in launch_dict.items() )
```

We can now export it to a CSV for the next section, but to make the answers consistent and in case you have difficulties finishing this lab.

Following labs will be using a provided dataset to make each lab independent.

```
df.to_csv('spacex_web_scraped.csv', index=False)
```

Authors

Yan Luo

Nayef Abou Tayoun

Copyright © 2021 IBM Corporation. All rights reserved.

```
-----
# TODO: Append the customer into launch_dict with key 'Customer'
customer = row[6].a.string
#print(customer)

# Launch outcome
# TODO: Append the launch_outcome into launch_dict with key 'Launch outcome'.
launch_outcome = list(row[7].strings)[0]
#print(launch_outcome)

# Booster Landing
# TODO: Append the launch_outcome into launch_dict with key 'Booster Landing'.
booster_landing = landing_status(row[8])
#print(booster_landing)
-----
```

After you have fill in the parsed launch record values into `launch_dict`, you can create a dataframe from it.

```
[ ]: df=pd.DataFrame(L_key:pd.Series(value) for key, value in launch_dict.items() )
```

We can now export it to a CSV for the next section, but to make the answers consistent and in case you have difficulties finishing this lab.

Following labs will be using a provided dataset to make each lab independent.

```
df.to_csv('spacex_web_scraped.csv', index=False)
```

Authors

Yan Luo

Nayef Abou Tayoun

Copyright © 2021 IBM Corporation. All rights reserved.

Would you like to receive official Jupyter news?
Please read the privacy policy.
[Open privacy policy](#) Yes No

Would you like to receive official Jupyter news?
Please read the privacy policy.
[Open privacy policy](#) Yes No